

Banco de Dados

Tipos de dados no SGBD MySql / MariaDB.

Professor Luís Carlos Pompeu

Introdução

- Tipos de dados são uma forma de classificar as informações que serão armazenados no banco de dados.
- Entender os tipos de dados que podem ser armazenados no banco e a que situações se adequam é importante para projetar um banco de dados o mais eficiente possível.

Introdução

- Antes de definir o tipo de dado, vale a pena prestar atenção nas seguintes diretrizes, para ajudar a ter melhores resultados:
 - quais os tipos de valores que podem ser usados;
 - que tamanhos podem ter;
 - as operações que podem ser realizadas;
 - seus significados;
 - se podem/serão indexados
 - como devem ser armazenados etc.

Dados numéricos

- **Signed x Unsigned:** basicamente, um número signed aceita dados negativos, enquanto o unsigned não.
- Isso cria o seguinte cenário: vamos supor que temos um dado do tipo TINYINT, que tem um range (faixa) de valores de 255, se ele for signed seu valor pode variar de **-128** a **127**, caso seja unsigned seu valor irá de **0** a **255**;

Dados numéricos

- Os tipos de dados que aceitam o atributo “UNSIGNED” automaticamente aceitam o atributo “SIGNED”.
- O padrão é **signed**, portanto esse atributo pode ser suprimido. Ex.: TINYINT é a mesma coisa que TINYINT SIGNED, caso for utilizá-lo como UNSIGNED a forma correta de escrita é TINYINT UNSIGNED

Dados numéricos

- Também é importante prestarmos atenção em alguns pontos:
- **ZEROFILL**: esse atributo é responsável por preencher com zeros à esquerda do valor informado sempre que utilizado, o dado automaticamente recebe o atributo UNSIGNED, ou seja, passa a não aceitar valores negativos. Ex.: em um TINYINT(4) ZEROFILL ao invés de armazenar o valor como “1”, seria armazenado como “001”;

Dados numéricos

- **Precisão simples x Precisão dupla:** um número de precisão simples contempla uma precisão de aproximadamente 7 casas decimais, enquanto que o de precisão dupla contempla aproximadamente 15 casas decimais. Este conceito se aplica a números de ponto flutuante.

Dados numéricos

- **BIT[(M)]:** como seu nome sugere, trata-se um dado do tipo bit, M indica o número de bits que o dado ocupará, podendo ser entre 1 e 64. Caso não seja informado, o valor padrão é 1.
- **TINYINT[(M)] [UNSIGNED] [ZEROFILL]:** um inteiro muito pequeno. Seu valor varia entre -128 e 127 caso seja signed, ou de 0 a 255 quando unsigned.

Paramos aqui em 29/05 – Turma 1

Dados numéricos

- **BOOL ou BOOLEAN:** na prática é o mesmo que usar um TINYINT(1). Quando 0, é considerado falso, quando 1 (ou outro valor diferente de 0), é considerado verdadeiro.
- **SMALLINT[(M)] [UNSIGNED] [ZEROFILL]:** um pouquinho maior que o TINYINT, seu valor varia de -8.388.608 a 8.388.607 ou 0 a 16.777.215, quando SIGNED ou UNSIGNED, respectivamente (acho que vocês já pegaram a ideia).

Dados numéricos

- **INT ou INTEGER[(M)] [UNSIGNED] [ZEROFILL]:** um inteiro de tamanho padrão, seu valor pode ser de - 2.147.483.648 a 2.147.483.647 ou de 0 a 4.294.967.295.

Dados numéricos

- **BIGINT[(M)] [UNSIGNED] [ZEROFILL]:** como vocês já devem imaginar, é um inteiro grande ou muito grande, seu range é -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807 ou 0 a 18.446.744.073.709.551.615.
- Como toda operação no MySQL é feita usando BIGINT ou DOUBLE, é recomendável que não se armazenem números maiores que 9.223.372.036.854.775.807 (63 bits), assim você evita resultados indesejados que podem ocorrer na conversão de BIGINT para DOUBLE.

Dados numéricos

- **DECIMAL[(M [, D])] [UNSIGNED] [ZEROFILL]**: um número de ponto fixo. *M* indica a quantidade total de dígitos (precisão) e *D* indica quantos deles estarão “depois da virgula”. O separador de decimais (.) e o indicador de números negativos (-) não são contados em *M*. No caso de o valor de *D* ser 0, o número não possuirá separador de decimais.
- Considere o seguinte: **valor decimal(10,2)**, isso significa que teremos 8 dígitos inteiros com 2 dígitos decimais e não 10 inteiros e 2 decimais.
- O valor máximo de *M* é 65 e de *D* é 30, caso omitidos, assumem os valores 10 e 0, respectivamente. Caso receba o atributo “UNSIGNED”, números negativos não serão suportados. Todas as operações com decimais acontecem com uma precisão (*M*) de 65 dígitos.

Dados numéricos

- **FLOAT[(M [, D])] [UNSIGNED] [ZEROFILL]:** um número de ponto flutuante de precisão simples. Os valores permitidos são de $-3.402823466E+38$ a $-1.175494351E-38$, 0, e de $1.175494351E-38$ a $3.402823466E+38$. Estes são limites teóricos baseados na norma técnica IEEE 754, mas seu range real é baseado no seu Hardware e Sistema Operacional, podendo ser menor.
- M indica a quantidade de dígitos e D indica quantos estarão após o separador decimal e, caso omitidos, os valores serão atribuídos de acordo com seu Hardware. Caso receba o atributo “UNSIGNED”, números negativos não serão suportados. Usar FLOAT pode acarretar em problemas inesperados, pois todo cálculo feito no MySQL utiliza números de precisão dupla.

Dados numéricos

- **DOUBLE[(M [, D])] [UNSIGNED] [ZEROFILL]:** número de ponto flutuante de precisão dupla. Os valores permitidos são de $-1.7976931348623157E+308$ até $-2.2250738585072014E-308$, 0, e $2.2250738585072014E-308$ até $1.7976931348623157E+308$. Todas as outras regras seguem as definições do tipo FLOAT. Também podem ser representados pelas seguintes formas:
 - DOUBLE PRECISION[(M [, D])] [UNSIGNED] [ZEROFILL];
 - REAL[(M [, D])] [UNSIGNED] [ZEROFILL]*.
- * Se o modo REAL_AS_FLOAT estiver habilitado, REAL representa um FLOAT ao invés de um DOUBLE.

tipo	tamanho	Range (Assinado)	Range (não assinado)	uso
TINYINT	1 byte	(-128.127)	(0255)	valores inteiros pequenos
SMALLINT	2 bytes	(768,32 -32 767)	(535 0,65)	valor inteiro
MEDIUMINT	3 bytes	(-8388 608,8 388 607)	(0,16 777.215)	valor inteiro
INT ou INTEGER	4 bytes	(-2 147 483 648,2 147 483 647)	(0,4 294 967 295)	valor inteiro
BIGINT	8 bytes	(-9.233.372.036.854.775 808 ,9 223.372.036.854.775 807)	(0,18 446.744.073.709.551 615)	valor máximo inteiro
FLOAT	4 bytes	(-3,402 823 466 E + 38,1.175 494 351 E-38), 0, (1.175 494 351 E-38,3.402 823 466 351 E + 38)	0, (1.175 494 351 E-38,3.402 823 466 E + 38)	valores de ponto flutuante de precisão simples
DOUBLE	8 bytes	(1,797 693 134 862 315 7 E + 308,2.225 073 858 507 201 4 E-308), 0, (2,225 073 858 507 201 4 E-308,1.797 693 134 862 315 7 E + 308)	0, (2,225 073 858 507 201 4 E-308,1.797 693 134 862 315 7 E + 308)	valores de ponto flutuante de precisão dupla
DECIMAL	De DECIMAL (H, D), se M> D, M + 2 é outra forma D + 2	Depende dos valores de M e D	Depende dos valores de M e D	valor decimal

Configuração de texto

- **CHARSET ou CHARACTER SET**
- Dependendo de como você criar seu Esquema (banco de dados) e, conseqüentemente, suas tabelas você verá algumas informações que podem ser confusas, o CHARSET é uma delas:
- Este atributo indica o conjunto de caracteres que você vai utilizar, para exemplificar, o mais utilizado atualmente é o UTF-8 que aceita 1.114.112 caracteres diferentes outro CHARSET conhecido é o ASCII, que suporta 256 caracteres diferentes.

Configuração de texto

- **COLLATE**

- Indica um “conjunto de regras” que define como o banco de dados vai se comportar nas buscas e organizações de strings.
- A nomenclatura das “collations” segue um padrão, como na tabela abaixo:

Sufixo	Significado
<code>_ai</code>	Accent insensitive
<code>_as</code>	Accent sensitive
<code>_ci</code>	Case insensitive
<code>_cs</code>	case-sensitive
<code>_ks</code>	Kana sensitive
<code>_bin</code>	Binary

Para “collations” do idioma japonês, o sufixo “_ks” indica que o banco vai diferenciar caracteres Katakana de Hiragana; caso o sufixo não seja utilizado, os caracteres não serão diferenciados para fins de ordenação. No caso do sufixo “_bin” as comparações e ordenações serão baseadas no valor em bytes das strings.

Tipos de dados em strings

- Strings são cadeias de caracteres. No MySQL, uma string pode ter qualquer conteúdo, desde texto simples a dados binários – tais como imagens e arquivos. Cadeias de caracteres podem ser comparadas e ser objeto de buscas.
- **CHAR[(M)]** — uma cadeia de caracteres (string), de tamanho fixo e não-binária, “M” pode assumir um valor entre 0 e 255 ;
- **VARCHAR[(M)]** — uma string de tamanho variável e não-binária. O valor de M pode variar de 0 a 65535, mas o valor máximo está intimamente ligado ao charset utilizado. ;
- **BINARY** — uma string binária de tamanho fixo;
- **VARBINARY** — uma string binária de tamanho variável;

tipo	tamanho	uso
CHAR	0-255 bytes	cadeia de comprimento fixo
VARCHAR	0-65535 bytes	cadeias de comprimento variável
TINYBLOB	0-255 bytes	Não mais do que 255 caracteres na cadeia binária
TINYTEXT	0-255 bytes	cadeias de texto curtas
BLOB	0-65535 bytes	dados longos de texto em formato binário
TEXT	0-65535 bytes	dados de texto longo
MEDIUMBLOB	0-16777215 bytes	forma binária de dados de texto de comprimento médio
MEDIUMTEXT	0-16777215 bytes	dados de texto de comprimento médio
LOB	0-4294967295 bytes	Grandes dados de texto em formato binário
LONGTEXT	0-4294967295 bytes	Grande dados de texto

tipo	tamanho (Byte)	escopo	formato	uso
DATA	3	1000/01/01 / 9999-12-31	AAAA-MM-DD	valores de data
TIME	3	'-838: 59: 59' / '838: 59: 59'	HH: MM: SS	valor do tempo o u a duração
ANO	1	1901/2155	AAAA	Valor ano
DATETIME	8	1000-01-0100: 00: 00 / 9999-12-31 23:59:59	AAAA-MM-DD H H: MM: SS	Mistura de data e hora valores
TIMESTAMP	4	Algum 00/2037 Ano: 1970-01-01 00:00	AAAAMMDD HH MMSS	Misturando data e valor de tempo, um timestamp

Para saber mais

- <https://elias.praciano.com/2014/01/mysql-tipos-de-dados/>
- <https://medium.com/mandabugs/mysql-tipos-de-dados-introdu%C3%A7%C3%A3o-e-dados-num%C3%A9ricos-1-de-3-a6e48fb5e1d3>
- <http://www.w3big.com/pt/mysql/mysql-data-types.html>
- <http://www.bosontreinamentos.com.br/mysql/mysql-tipos-de-dados-comuns-09/>